

The Convex Hull Trick

Theory, Implementations, and a Full Solution to *Points and Threads*

Lecture Notes

1 The Problem

We have n points on a line. Point i has position x_i and cost rate a_i . Joining i and j directly costs

$$w(i, j) = \min(a_i, a_j) |x_i - x_j|.$$

We must connect all points with $n - 1$ threads at minimum total cost. This is exactly a **minimum spanning tree (MST)** of the complete graph K_n with weights w .

Kruskal or Prim run directly on K_n need $\Theta(n^2)$ edges, which is far too slow for $n = 2 \cdot 10^5$. Our plan has three parts:

- (i) show that some MST uses only $O(n)$ candidate edges;
- (ii) find those candidates with a *Convex Hull Trick* data structure;
- (iii) run Kruskal on the candidates.

Part (ii) is where the Convex Hull Trick comes in. We study it in general first, because it applies to many other problems.

2 The Convex Hull Trick (CHT)

2.1 The abstract problem

Definition 1 (Line-minimum problem). We keep a set $\mathcal{L}$ of lines $\ell_k(x) = m_k x + b_k$ and support two operations:

- INSERT(m, b): add the line $y = mx + b$ to $\mathcal{L}$;
- QUERY(x): return $\min_{\ell \in \mathcal{L}} \ell(x)$, and often also the line that achieves it.

The naive method scans all lines, so it costs $O(|\mathcal{L}|)$ per query. The CHT answers each query in $O(\log n)$, or in amortised $O(1)$ in special cases.

2.2 The lower envelope

Definition 2. The *lower envelope* of $\mathcal{L}$ is the function $E(x) = \min_{\ell \in \mathcal{L}} \ell(x)$.

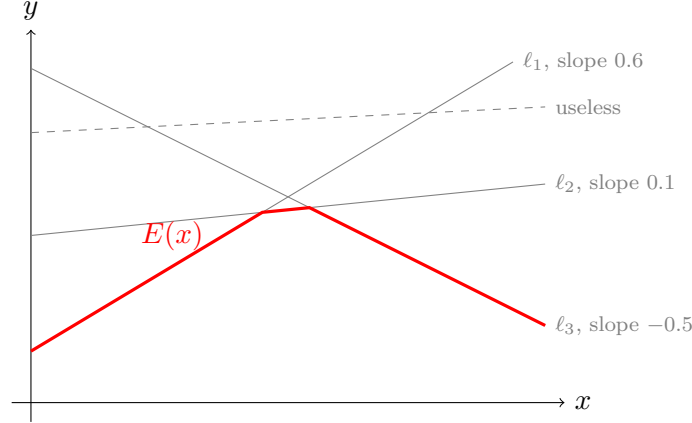
Proposition 3. E is concave and piecewise linear. As x increases from $-\infty$ to $+\infty$, the lines that appear on the envelope occur in strictly decreasing order of slope. Each line appears on at most one contiguous interval.

Proof. A minimum of affine functions is concave, and it is piecewise linear with finitely many pieces.

Take two lines with slopes $m_p > m_q$. Their difference $\ell_p - \ell_q$ is strictly increasing. So $\ell_p < \ell_q$ holds exactly on an interval of the form $(-\infty, c)$. In words, the steeper line can beat the flatter one only to the left of their intersection.

Therefore, if ℓ_p is optimal at x_1 and ℓ_q is optimal at $x_2 > x_1$, we must have $m_p \geq m_q$.

The set where a fixed line is minimal is $\bigcap_q \{x : \ell_p(x) \leq \ell_q(x)\}$. This is an intersection of intervals, hence itself an interval, which proves contiguity. $\square$



A line that never appears on the envelope, such as the dashed line in the figure, is *useless*. We can delete it permanently.

2.3 Why it is called a “convex hull” trick: point–line duality

Map each line $\ell(x) = mx + b$ to the point $P = (m, b)$. For a fixed query x ,

$$\ell(x) = mx + b = \langle (x, 1), (m, b) \rangle.$$

So $\text{QUERY}(x)$ asks for the point P that minimises a linear functional in the direction $(x, 1)$. A linear functional over a finite point set is always minimised at a vertex of the convex hull. Because the second coordinate of the direction is positive, that vertex lies on the *lower* convex hull of the points $\{(m_k, b_k)\}$.

The two views match exactly: the lines that are useful on the envelope correspond to the points on the lower hull. Keeping the envelope is the same as keeping a convex hull, and that is where the name comes from.

2.4 The removal test

Suppose we store the lines of the envelope sorted by decreasing slope, and the last two are ℓ_1, ℓ_2 with $m_1 > m_2$. A new line ℓ_3 arrives with $m_3 < m_2$. Then ℓ_2 is useless if and only if ℓ_3 overtakes ℓ_1 no later than ℓ_2 does:

$$X(\ell_1, \ell_3) \leq X(\ell_1, \ell_2), \quad X(\ell_p, \ell_q) = \frac{b_q - b_p}{m_p - m_q}.$$

Clearing the positive denominators gives a test that uses only integers:

$$(b_3 - b_1)(m_1 - m_2) \leq (b_2 - b_1)(m_1 - m_3). \quad (1)$$

The products can reach about 10^{36} when slopes and intercepts are near 10^{18} . In that case, use `__int128`.

3 Implementation 1: Monotone CHT (deque)

This version applies when **lines are inserted in decreasing slope order**. If the **queries are also non-decreasing in x** , each query costs amortised $O(1)$. With arbitrary query order, use a binary search instead, which costs $O(\log n)$.

```

1 typedef long long ll;
2 struct Line { ll m, b; ll eval(ll x) const { return m * x + b; } };
3
4 struct MonotoneCHT {                                // minimum; slopes inserted strictly decreasing
5     std::deque<Line> dq;
6     static bool bad(const Line& l1, const Line& l2, const Line& l3) {
7         // equation (bad): l2 is useless once l3 is added
8         return (__int128)(l3.b - l1.b) * (l1.m - l2.m)
9             <= (__int128)(l2.b - l1.b) * (l1.m - l3.m);
10    }
11    void insert(ll m, ll b) {
12        Line nw{m, b};
13        if (!dq.empty() && dq.back().m == m) {          // equal slopes: keep lower b
14            if (dq.back().b <= b) return;
15            dq.pop_back();
16        }
17        while (dq.size() >= 2 && bad(dq[dq.size()-2], dq.back(), nw))
18            dq.pop_back();
19        dq.push_back(nw);
20    }
21    ll query(ll x) {                                    // x non-decreasing over calls
22        while (dq.size() >= 2 && dq[0].eval(x) >= dq[1].eval(x))
23            dq.pop_front();
24        return dq.front().eval(x);
25    }
26 };

```

Why the pops are safe. The `pop_back` step is correct by the test (1).

For `pop_front`: by Proposition 3, once ℓ_1 beats ℓ_0 at some x , it keeps beating ℓ_0 for every larger x . Since later queries are at least as large, ℓ_0 is never needed again.

Every line is pushed once and popped at most once. So q queries and n insertions cost $O(n+q)$ in total.

4 Implementation 2: Li Chao Tree (fully dynamic)

In our problem the slopes a_i arrive in *arbitrary* order, so the deque version does not apply. The Li Chao tree handles this case. It supports insertion of lines in any order and queries at any x from a known finite set of coordinates $u_0 < u_1 < \dots < u_{M-1}$. Each operation costs $O(\log M)$.

4.1 Idea

Build a segment tree over the index range $[0, M-1]$. Every node covering $[l, r]$ with midpoint m stores at most one line: the line that is best at u_m among those that reached the node.

To insert a line f at a node that currently stores g :

1. Compare f and g at u_m . Keep the better one at the node, and call the other one the loser.
2. Two lines cross at most once. So the loser can be better than the winner on at most one side of m .
3. Compare the two lines at u_l to find that side. Push the loser only into that child.

Lemma 4 (Li Chao invariant). *For every coordinate u_p , the best line at u_p is stored at some node on the root-to-leaf path of p .*

Proof. We use induction on insertions. When a line is discarded from a node's subtree, it is worse than the kept line at u_m . If it is also worse at u_l , then by the single-crossing property it is worse on the whole of $[u_l, u_m]$. The only region where it could still be better is the side it is pushed into.

The symmetric argument covers the other case. So no line is ever removed from a region where it could be optimal. $\square$

Therefore a query walks from the root to leaf p and takes the minimum over the $O(\log M)$ lines stored along the path.

4.2 Code (returns value and argmin index)

```

1 typedef long long ll;
2 struct LLine { ll a, x0; int id; };           // value(x) = a*(x - x0)
3 inline ll evalL(const LLine& L, ll x) { return L.a * (x - L.x0); }
4
5 struct LiChao {                               // minimum over lines, queries at xs[p]
6     std::vector<ll> xs; std::vector<LLine> t; std::vector<char> has;
7     void init(const std::vector<ll>& coords) {
8         xs = coords; t.assign(4 * xs.size(), {0, 0, -1}); has.assign(4 * xs.size(), 0);
9     }
10    void insert(LLine nw, int node, int l, int r) {
11        if (!has[node]) { t[node] = nw; has[node] = 1; return; }
12        int m = (l + r) / 2;
13        bool lef = evalL(nw, xs[l]) < evalL(t[node], xs[l]);
14        bool mid = evalL(nw, xs[m]) < evalL(t[node], xs[m]);
15        if (mid) std::swap(t[node], nw);       // node keeps winner at xs[m]
16        if (l == r) return;
17        if (lef != mid) insert(nw, 2*node, l, m); // loser may win on left
18        else insert(nw, 2*node+1, m+1, r);     // loser may win on right
19    }
20    std::pair<ll,int> query(int p, int node, int l, int r) {
21        std::pair<ll,int> res = has[node]
22            ? std::make_pair(evalL(t[node], xs[p]), t[node].id)
23            : std::make_pair(LLONG_MAX, -1);
24        if (l == r) return res;
25        int m = (l + r) / 2;
26        auto sub = (p <= m) ? query(p, 2*node, l, m) : query(p, 2*node+1, m+1, r);
27        return std::min(res, sub);
28    }
29 };

```

Remark 5. We write each line as $a(x - x_0)$ and not as $ax + b$. With $b = -ax_0$ and x close to x_0 , the expanded form computes two terms of size about 10^{18} and subtracts them. The factored form computes a times a difference of at most 10^9 , which stays well inside the range of `long long`.

4.3 Choosing the right CHT variant

Slopes inserted	Queries	Tool, cost per op
monotone	monotone	deque CHT, amortised $O(1)$
monotone	arbitrary	deque CHT + binary search, $O(\log n)$
arbitrary	arbitrary, coordinates known	Li Chao tree, $O(\log M)$
arbitrary	arbitrary, online	dynamic hull (“LineContainer”), $O(\log n)$

5 Solving *Points and Threads*

5.1 A tempting idea that is wrong

One natural guess is this: each point connects to the cheapest thread it can reach among points with a smaller a . The resulting edges form a tree, but that tree is *not* always minimal.

Example 6. Take $x = (1, 101, 102)$ and $a = (1, 5, 2)$. The pairwise weights are $w(1, 2) = 100$, $w(1, 3) = 101$ and $w(2, 3) = 2$.

The greedy rule attaches point 3 to point 1 at cost 101, because point 1 is the only point with a smaller a . It attaches point 2 to point 3 at cost 2. The total is 103.

The true MST is $\{(1, 2), (2, 3)\}$ with weight 102. The constraint “smaller a ” is the wrong one to impose.

5.2 The candidate-edge lemma

Sort the points by x . For each point i , define two lines in the variable x :

$$L_i(x) = a_i(x - x_i), \quad R_i(x) = a_i(x_i - x).$$

$L_i(x)$ is the cost of a thread bought at i that reaches a point at x to the right of i . $R_i(x)$ is the same for a point to the left. For every point j define:

$$\lambda(j) = \arg \min_{i \text{ left of } j} L_i(x_j), \quad \rho(j) = \arg \min_{i \text{ right of } j} R_i(x_j).$$

Note that there is **no constraint** relating a_i and a_j . The candidate set is

$$\mathcal{C} = \{(j, \lambda(j))\} \cup \{(j, \rho(j))\}, \quad |\mathcal{C}| \leq 2n - 2.$$

Theorem 7. Assume the positions are distinct and there are no ties among the relevant costs. Then every MST of K_n uses only edges of $\mathcal{C}$.

Proof. We use the *cycle property*: an edge that is strictly the heaviest edge on some cycle of G belongs to no MST.

Let $e = (i, j)$ be an edge with $x_i < x_j$ that is not in $\mathcal{C}$. There are two cases.

Case $a_i \leq a_j$. Here $w(e) = a_i(x_j - x_i) = L_i(x_j)$. Since $e \notin \mathcal{C}$, we have $i \neq \lambda(j)$. Let $k = \lambda(j)$. Then $x_k < x_j$ and

$$L_k(x_j) < L_i(x_j) = w(e).$$

We show that both other edges of the triangle i, j, k are strictly lighter than e .

- Edge (j, k) : $w(j, k) = \min(a_j, a_k)(x_j - x_k) \leq a_k(x_j - x_k) = L_k(x_j) < w(e)$.
- Edge (i, k) when $x_i < x_k < x_j$: $w(i, k) \leq a_i(x_k - x_i) < a_i(x_j - x_i) = w(e)$.
- Edge (i, k) when $x_k < x_i$: $w(i, k) \leq a_k(x_i - x_k) < a_k(x_j - x_k) = L_k(x_j) < w(e)$.

So e is strictly the heaviest edge on the triangle (i, j, k) , and it lies in no MST.

Case $a_i > a_j$. Here $w(e) = a_j(x_j - x_i) = R_j(x_i)$. Apply the mirror image of the previous argument with $k = \rho(i)$, reflecting the line through $x \mapsto -x$. $\square$

Remark 8 (Ties). When positions repeat or costs tie, some of the strict inequalities above become non-strict. The conclusion weakens to “some MST lies in $\mathcal{C}$ ”, which is all the algorithm needs. The standard way to see this is to perturb the inputs infinitesimally so that all ties are broken.

As a sanity check, the program below was compared against an $O(n^2)$ Prim implementation on 10^5 random tests with heavy ties, all small values, and on 10^3 tests with $n = 200$ and values up to 10^9 . It matched on every test.

Remark 9 (Why the result is intuitive). The lemma says each point only needs to consider the single cheapest thread arriving from its left and the single cheapest arriving from its right. We ignore whose a is smaller when choosing $\lambda(j)$.

This works because if the true minimum uses a_j instead, then point k is *closer* to j than the competing point is. The edge (j, k) is then even cheaper, and it is already in $\mathcal{C}$.

5.3 Finding $\lambda(j)$ with CHT

Sweep the points in increasing order of x . Before processing j , the structure holds the lines L_i of all points to its left. We query at x_j to get $\lambda(j)$, and then insert L_j .

The slopes a_i arrive in arbitrary order, so we use a **Li Chao tree** over the sorted distinct positions.

For $\rho(j)$, reflect all positions ($x \mapsto -x$) and sweep again. The line $R_i(x) = a_i(x_i - x)$ becomes $a_i((-x) - (-x_i))$, which has the same form as L_i . So the same code works unchanged.

Every candidate edge is stored with its *true* weight $\min(a_i, a_j)|x_i - x_j|$. We do not store the line value. Finally, Kruskal with a DSU runs on the at most $2n$ candidates.

5.4 Worked example (the sample)

The input is $x = (1, 3, 4)$, $a = (1, 11, 111)$.

Left sweep:

- $j = 2$ ($x = 3$): the only line is $L_1 = 1 \cdot (x - 1)$, with value 2. So $\lambda(2) = 1$, giving edge $(1, 2)$ with weight 2.
- $j = 3$ ($x = 4$): $L_1(4) = 3$ and $L_2(4) = 11 \cdot 1 = 11$. So $\lambda(3) = 1$, giving edge $(1, 3)$ with weight $\min(1, 111) \cdot 3 = 3$.

Right sweep:

- $j = 2$: $R_3(3) = 111$. So $\rho(2) = 3$, giving edge $(2, 3)$ with weight $\min(11, 111) \cdot 1 = 11$.
- $j = 1$: $R_3(1) = 333$ and $R_2(1) = 22$. So $\rho(1) = 2$, giving edge $(1, 2)$ with weight 2.

Kruskal on the sorted candidates 2, 2, 3, 11:

- take $(1, 2)$ with weight 2;
- skip the duplicate $(1, 2)$;
- take $(1, 3)$ with weight 3.

The answer is 5, as expected.

5.5 Complexity and overflow

- *Time*. Sorting costs $O(n \log n)$. The two sweeps make $2n$ Li Chao operations at $O(\log n)$ each. Kruskal sorts $\leq 2n$ edges. The total is $O(n \log n)$, which ran in about 0.26 s for $n = 2 \cdot 10^5$.
- *Memory*. $O(n)$.
- *Range*. Any single weight is at most $10^9 \cdot 10^9 = 10^{18}$. The MST weight is at most the weight of the “chain of neighbours” tree, which is $\leq \max a \cdot (\max x - \min x) < 10^{18}$. So signed 64-bit integers suffice, since `LLONG_MAX` $\approx 9.2 \cdot 10^{18}$.

5.6 Full solution

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 struct LLine { ll a, x0; int id; }; // value(x) = a*(x - x0)
6 static inline ll evalL(const LLine& L, ll x) { return L.a * (x - L.x0); }
7
8 struct LiChao {
9     vector<ll> xs; vector<LLine> t; vector<char> has;
10     void init(const vector<ll>& c) { xs = c; t.assign(4*xs.size(), {0,0,-1}); has.assign(4*xs.size(), 0); }
11     void insert(LLine nw, int node, int l, int r) {
12         if (!has[node]) { t[node] = nw; has[node] = 1; return; }
13         int m = (l + r) / 2;
14         bool lef = evalL(nw, xs[l]) < evalL(t[node], xs[l]);
15         bool mid = evalL(nw, xs[m]) < evalL(t[node], xs[m]);
16         if (mid) swap(t[node], nw);
17         if (l == r) return;
18         if (lef != mid) insert(nw, 2*node, l, m); else insert(nw, 2*node+1, m+1, r);
19     }
20     pair<ll,int> query(int p, int node, int l, int r) {
21         pair<ll,int> res = has[node] ? make_pair(evalL(t[node], xs[p]), t[node].id)
22                                     : make_pair(LLONG_MAX, -1);
23         if (l == r) return res;
24         int m = (l + r) / 2;
25         auto sub = (p <= m) ? query(p, 2*node, l, m) : query(p, 2*node+1, m+1, r);
26         return min(res, sub);
27     }
28 };
29
30 struct DSU {
31     vector<int> p;
32     DSU(int n) : p(n) { iota(p.begin(), p.end(), 0); }
33     int f(int x) { while (p[x] != x) x = p[x] = p[p[x]]; return x; }
34     bool unite(int a, int b) { a = f(a); b = f(b); if (a == b) return false; p[a] = b; return true; }
35 };
36
37 // For each point j in 'ord' (sorted by X ascending), add edge (j, argmin_{i before j} A[i]*(X[j]-X[i]))
38 void sweep(const vector<ll>& X, const vector<ll>& A, const vector<int>& ord,
39           vector<tuple<ll,int,int>>& E) {
40     int n = ord.size();
41     vector<ll> u(n);
42     for (int k = 0; k < n; k++) u[k] = X[ord[k]];
43     u.erase(unique(u.begin(), u.end()), u.end());
44     LiChao lc; lc.init(u); int M = u.size();
45     for (int k = 0; k < n; k++) {
46         int j = ord[k];
47         int p = lower_bound(u.begin(), u.end(), X[j]) - u.begin();
48         if (k > 0) {
49             int i = lc.query(p, 1, 0, M-1).second; // lambda(j)
50             E.emplace_back(min(A[i], A[j]) * llabs(X[i] - X[j]), i, j); // TRUE weight
51         }
52         lc.insert({A[j], X[j], j}, 1, 0, M-1); // line L_j
53     }
54 }

```

```

55
56 int main() {
57     int t;
58     if (scanf("%d", &t) != 1) return 0;
59     while (t--) {
60         int n; if (scanf("%d", &n) != 1) return 0;
61         vector<ll> X(n), A(n);
62         for (auto& v : X) if (scanf("%lld", &v) != 1) return 0;
63         for (auto& v : A) if (scanf("%lld", &v) != 1) return 0;
64
65         vector<int> ord(n); iota(ord.begin(), ord.end(), 0);
66         sort(ord.begin(), ord.end(), [&](int p, int q) { return X[p] < X[q]; });
67
68         vector<tuple<ll,int,int>> E;
69         sweep(X, A, ord, E); // lambda edges
70         vector<ll> NX(n); for (int i = 0; i < n; i++) NX[i] = -X[i];
71         vector<int> rord(ord.rbegin(), ord.rend());
72         sweep(NX, A, rord, E); // rho edges (mirror)
73
74         sort(E.begin(), E.end());
75         DSU d(n); ll ans = 0;
76         for (auto& [w, i, j] : E) if (d.unite(i, j)) ans += w;
77         printf("%lld\n", ans);
78     }
79 }

```

6 Recognising CHT in Other Problems

6.1 The general pattern

Look for a recurrence or an optimisation of the form

$$\text{best}(j) = \min_{i \in S(j)} \left(\underbrace{m_i}_{\text{depends on } i} \cdot \underbrace{q_j}_{\text{depends on } j} + \underbrace{b_i}_{\text{depends on } i} \right) + c_j$$

The recipe is: expand the cost, and put every term that mixes i and j into a single product $m_i q_j$. Each i then becomes a line with slope m_i and intercept b_i , and each j becomes a query at the point q_j .

6.2 Classic example: DP with a squared cost

Consider

$$dp_j = \min_{i < j} (dp_i + (P_j - P_i)^2 + C),$$

where P is an increasing prefix sum. Expanding the square gives

$$dp_j = P_j^2 + C + \min_{i < j} ((-2P_i) P_j + (dp_i + P_i^2)).$$

So each i is the line with slope $-2P_i$ and intercept $dp_i + P_i^2$, queried at $x = P_j$. The slopes decrease and the queries increase, so the deque CHT gives $O(n)$ instead of $O(n^2)$.

6.3 Lessons from *Points and Threads*

1. **Graph problems can hide CHT.** First reduce the MST on K_n to $O(n)$ candidate edges using exchange or cycle arguments. Then find each candidate with a line-minimum query.

2. **Remove min / max carefully.** Replacing $\min(a_i, a_j)$ by a_i gives an *upper bound* on the weight. The proof showed that this relaxation still finds every needed edge, as long as candidate edges are stored with their true weights.
3. **Absolute values mean two sweeps.** Split $|x_i - x_j|$ into a left case and a right case. Reflecting $x \mapsto -x$ lets one routine handle both.
4. **Keep the argmin.** When the CHT output selects an *object* such as an edge or a transition, store the line identifier together with its coefficients.

6.4 Exercises

1. Change the problem so that the cost is $\max(a_i, a_j)|x_i - x_j|$. Does a candidate lemma still hold? (Hint: the nearest neighbours in x and the triangle inequality.)
2. Solve $dp_j = \min_{i < j}(dp_i + a_i \cdot b_j)$ when both a_i and b_j are arbitrary. Which variant from the table do you need?
3. Prove that the MST weight is at most $\max a \cdot (\max x - \min x)$. Use this to justify 64-bit arithmetic.
4. Rewrite the monotone deque CHT for *maximum* queries. What changes in (1)?
5. Classic problems to practise on: *APIO 2010 Commando*, *CEOI 2017 Building Bridges* (Li Chao tree), *Codeforces 319C Kalila and Dimna*, *Codeforces 1083E The Fair Nut and Rectangles*.